Worksheet 2: Build and Break

RAG pipeline -- build it, test it, try to fool it

Building AI Agents Workshop | SRM AP x Kalvium | November 2026

An AI that sounds confident but makes things up is worse than no AI at all. In this worksheet, you'll build a RAG system over YOUR content, rigorously test its boundaries, and discover exactly where it breaks. The goal isn't a perfect system -- it's understanding WHY it works and WHY it fails.

A. Choose your knowledge domain

Pick content you actually know well -- so YOU can judge whether the AI's answers are correct.

My domain (e.g., DSA notes, club FAQ, recipe collection, song lyrics):

Why I chose this (what makes it a good test):

B. Write your knowledge base

Write 8-10 chunks of knowledge. One clear fact or concept per line. Specific beats vague.

Good: 'Stacks follow LIFO. Push and Pop are O(1). Used in function calls and undo.'

Bad: 'Stacks are a data structure.'

Chunk 1: _____

Chunk 2: _____

Chunk 3: _____

Chunk 4: _____

Chunk 5: _____

Chunk 6:

Chunk 7: _____

Chunk 8: _____

Chunk 9: _____

Chunk 10: _____

C. Load and test -- the happy path

Load your chunks into the RAG demo (Slide 11). Ask 3 questions that SHOULD be answerable.

Question	Correct? (Y/N + notes)
Q1:	
Q2:	
Q3:	

For the best answer: which chunk(s) were retrieved? Were they the right ones?

D. Break it -- the adversarial path

Now ask 3 questions that should NOT be answerable. Your goal: make the AI hallucinate.

Trick question	Refused gracefully? Or hallucinated?
Q1:	
Q2:	
Q3:	

If it hallucinated: what went wrong? Wrong chunk retrieved, or did it ignore the 'only answer from notes' rule?

E. The edge cases

The interesting cases -- questions that are PARTIALLY covered by your notes.

A question where only HALF the answer is in your notes. What happened?

A question using different WORDS but same MEANING as your notes. Did embeddings catch it?

An ambiguous question that could match multiple chunks. Which ones were retrieved?

F. Improve your knowledge base

Based on failures, rewrite or add chunks to fix them.

Chunks I changed or added (and why):

Did the improvements fix the failures? What's still broken?

G. Reflection: trust and AI

RAG is how companies deploy AI they can trust. But trust is earned through testing, not assumed.

Would you trust this system to answer questions for someone who DOESN'T know the subject? Why or why not?

What would need to change before you'd deploy this for real users?

One thing that surprised you about how embeddings matched (or didn't match) your queries: